

Task 2

AI Tic-Tac-Toe Game using Minimax Algorithm

Submitted by

Pavani S K Reddy

Table of Contents

1. Introduction
2. Objective
3. Project Overview
4. Technologies Used
5. System Architecture
6. Project Modules
7. Minimax Algorithm
8. Implementation
9. Features
10. Screenshots
11. Learning Outcomes
12. Conclusion

1. Introduction

Artificial Intelligence has become one of the most influential fields in modern software development. This project focuses on implementing an intelligent Tic-Tac-Toe game using Python, where a human player competes against an AI opponent. The AI makes optimal decisions using the Minimax algorithm, ensuring challenging gameplay.

2. Objective

The objective of this project is to develop an intelligent Tic-Tac-Toe game that demonstrates fundamental Artificial Intelligence concepts. The AI should always make the best possible move using the Minimax search algorithm while maintaining a modular and reusable Python codebase.

3. Project Overview

The application is a console-based Tic-Tac-Toe game developed in Python. It provides an interactive interface where a user plays against an AI opponent. The AI evaluates all possible game states before selecting the optimal move.

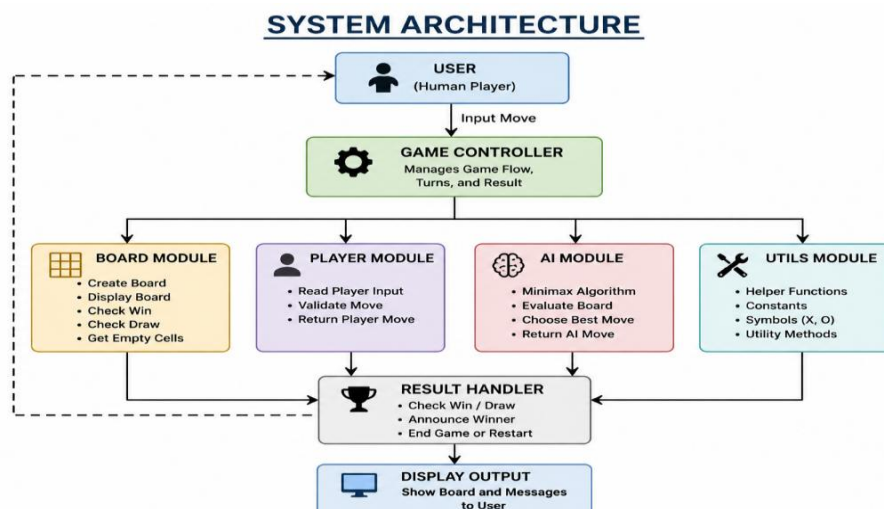
The project emphasizes:

- Artificial Intelligence
- Game Theory
- Search Algorithms
- Object-Oriented Programming
- Modular Design

4. Technologies Used

Technology	Purpose
Python 3	Programming Language
VS Code	Development Environment
Git	Version Control
GitHub	Source Code Hosting
Minimax Algorithm	AI Decision Making

5. System Architecture



6. Project Modules

board.py

Handles:

- Board creation
- Displaying board
- Win detection
- Draw detection
- Empty cell tracking

player.py

Responsible for:

- Reading player moves
- Validating user input

ai.py

Implements:

- Minimax Algorithm
- Best move calculation
- AI move selection

game.py

Controls

- Game loop
- Switching turns
- Winner checking
- Draw handling

main.py

Entry point of the application.

7. Minimax Algorithm

The AI opponent uses the Minimax Algorithm to evaluate every possible future game state.

Working process:

1. Generate every possible move.
2. Simulate future game states.
3. Assign scores.
4. Choose the move with the highest score.
5. Repeat after every player move.

Scoring:

AI Win = +1

Player Win = -1

Draw = 0

Since Tic-Tac-Toe has a small search space, Minimax guarantees the optimal move.

8. Features

- Human vs AI gameplay
- Intelligent AI using Minimax
- Input validation
- Win detection
- Draw detection
- Clean console interface
- Modular source code
- Easy to maintain

9. Project Folder Structure

Task2_TicTacToeAI/

- | — ai.py
- | — board.py
- | — game.py
- | — player.py
- | — utils.py
- | — main.py
- | — requirements.txt
- | — README.md
- | — screenshots/

10. Screenshots

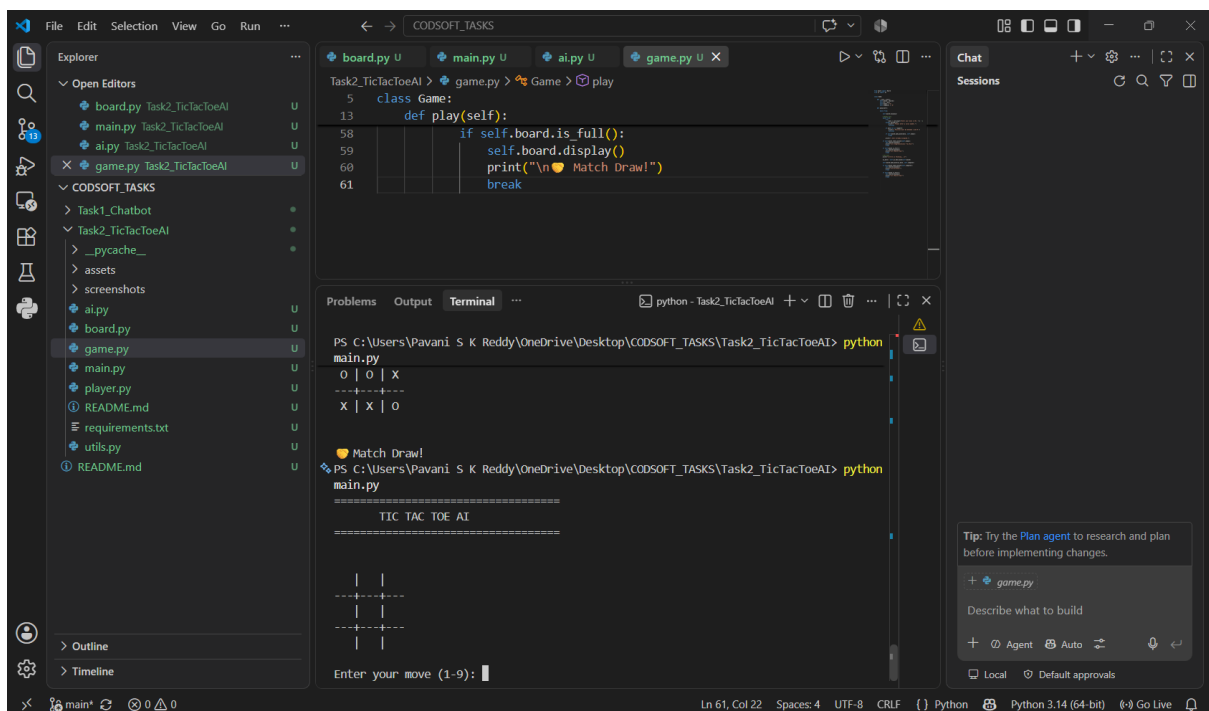


Figure 1: Game Start

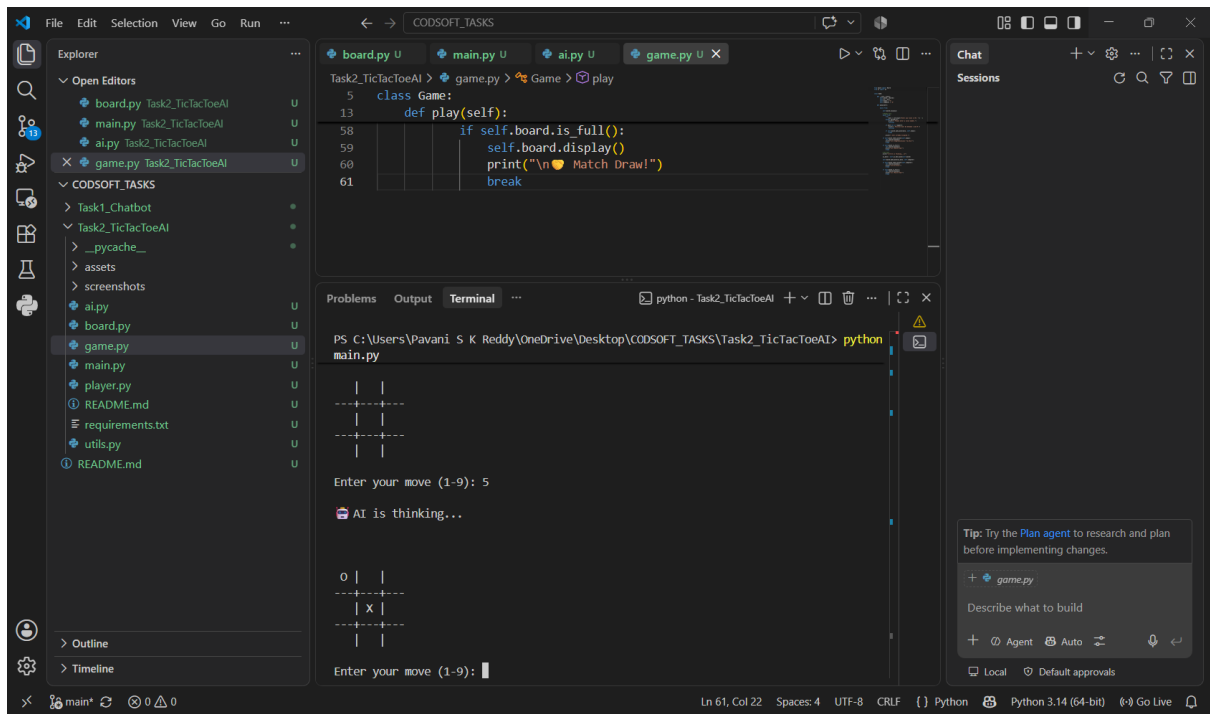


Figure 2: AI Thinking

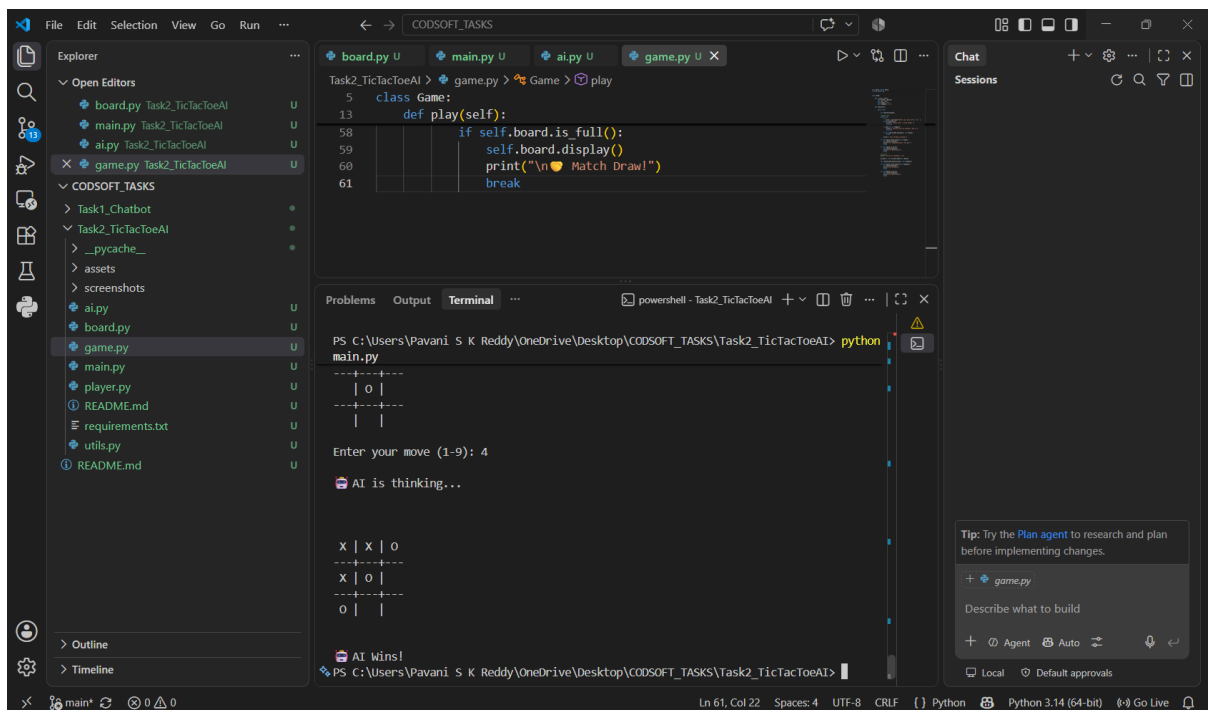


Figure 3: AI Wins

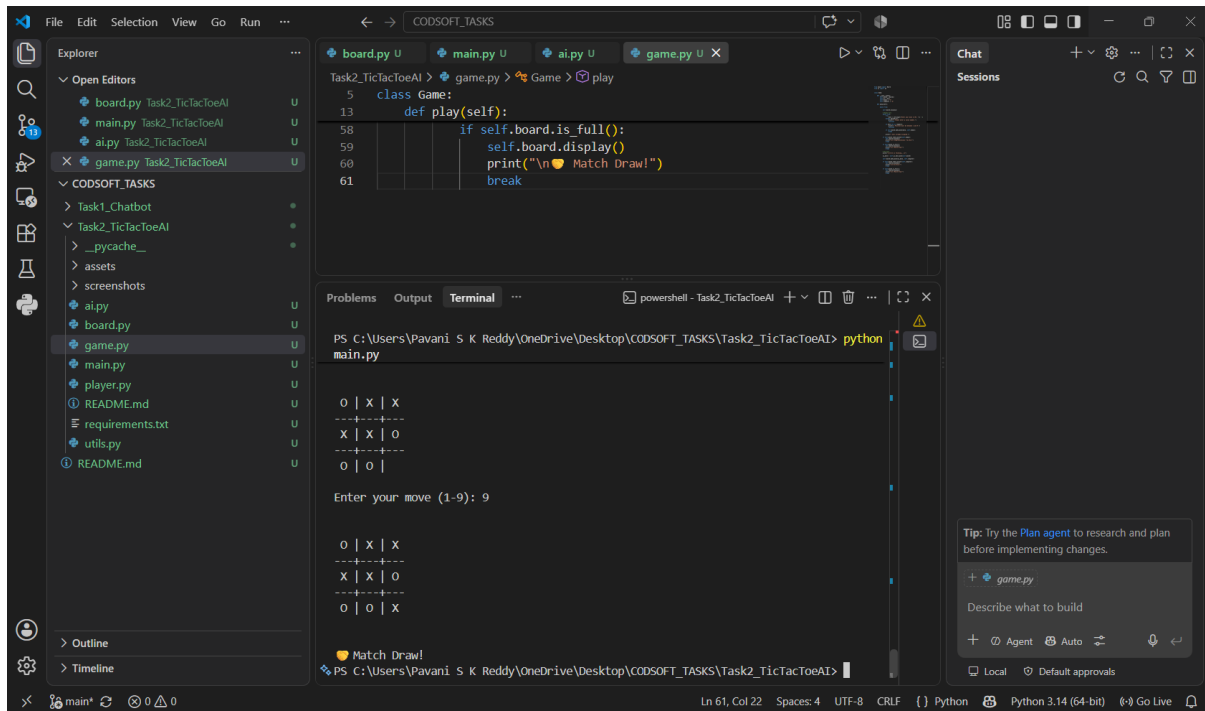


Figure 4: Match Draw

11. Learning Outcomes

During this project, I learned:

- Artificial Intelligence fundamentals
- Game Theory
- Minimax Search Algorithm
- Recursive programming
- Object-Oriented Programming
- Python modular development
- Git and GitHub project management

12. Conclusion

The AI Tic-Tac-Toe project successfully demonstrates the application of Artificial Intelligence in game development. The Minimax algorithm enables the AI to make optimal decisions, resulting in an unbeatable opponent. The project also strengthened practical knowledge of Python programming, algorithm design, software modularization, and version control using Git and GitHub.

